Working Remotely

SSH Setup

Harald Schernthanner

2026-07-01

Table of contents

1 Contact **1**

2 Objectives **2**

3 Working Remotely: PC Pool in Golm (Building 32) **2**
- 3.1 Login Via `ssh` . . . 2
  - 3.1.1 SSH Key Generation and Login: Windows . . . 2
  - 3.1.2 `ssh` Key Generation and Login: Linux / Mac . . . 5
- 3.2 Data Storage on PC Pool Computers . . . 6
- 3.3 File Transfer Between Local and Remote Computers . . . 6
  - 3.3.1 GUI: FileZilla - *Your most likely choice* . . . 6
  - 3.3.2 Command Line: `rsync` . . . 8

4 Using JupyterLab Remotely Via SSH **9**
- 4.1 Using `screen` Sessions . . . 9
- 4.2 Opening Remote Notebook Via Port Forwarding: Linux / Mac . . . 10
- 4.3 Opening Remote Notebook Via Port Forwarding: Windows . . . 11

1 Contact

If you have questions about this document you can find our team webpage with contact info at https://up-rs-esp.uni-potsdam.de/. Most questions can be addressed to Dr. Harald Schernthanner (hschernt@uni-potsdam.de).

2 Objectives

This document describes in detail how to access and work remotely using the PC Pool at Potsdam using `ssh` logins. You should already be familiar with your command line, setting up a Python environment, and Jupyter notebooks from the other manual titled *Geological Remote Sensing Python Setup*.

3 Working Remotely: PC Pool in Golm (Building 32)

If you do not or cannot install the required software or Python environment on your personal computer, we have the option of working remotely through the PC Pool at Campus Golm. Heavy computation may also require the more powerful PC Pool on campus, so it's good to familiarize yourself with these steps and generate your own SSH private/public key pair for login to the PC Pool.

3.1 Login Via `ssh`

SSH stands for secure shell and is a safe and encrypted way to access computers remotely. We will work with the `ssh` command-line environment. The below steps will walk you through ssh private/public key generation, allowing secure and easy login without passwords. Once logged into the remote PC Pool computer you will be in a Linux (e.g. Ubuntu) command-line environment.

3.1.1 SSH Key Generation and Login: Windows

Modern Windows systems include the OpenSSH client, or it can be installed as an optional Windows feature. Therefore, Windows users can usually work directly from **PowerShell** or **Windows Terminal** without installing additional SSH software.

If you prefer a graphical SSH client, you may still use PuTTY. See the separate section **Alternative: Using PuTTY on Windows** below.

3.1.1.1 Recommended method: OpenSSH in PowerShell or Windows Terminal

1. Open **PowerShell** or **Windows Terminal** and check whether SSH is available:

```
ssh -V
```

2. If SSH is not available, install the **OpenSSH Client** via Windows Settings:

   Settings > System > Optional features > Add an optional feature > OpenSSH Client

3. Generate a new SSH key pair. We recommend using an **Ed25519** key:

```
mkdir $env:USERPROFILE\.ssh -Force
ssh-keygen -t ed25519 -a 100 -f $env:USERPROFILE\.ssh\pcpool_ed25519 -C "your.name@uni-potsda
```

If Ed25519 is not available, generate an RSA key with at least 4096 bits instead:

```
ssh-keygen -t rsa -b 4096 -a 100 -f $env:USERPROFILE\.ssh\pcpool_rsa -C "your.name@uni-potsda
```

4. When asked for a passphrase, choose a strong passphrase. This protects your private key if your computer is lost or compromised.

5. The command creates two files in your `.ssh` folder:

```
C:\Users\<your-windows-username>\.ssh\pcpool_ed25519
C:\Users\<your-windows-username>\.ssh\pcpool_ed25519.pub
```

The file without `.pub` is your **private key**. Keep it secure and never share it with anyone. The file ending in `.pub` is your **public key**. Send only this public key to the system administrator, Dr. Harald Schernthanner (hschernt@uni-potsdam.de).

You can display the public key in PowerShell with:

```
Get-Content $env:USERPROFILE\.ssh\pcpool_ed25519.pub
```

Copy the complete output and send it by email.

6. After receiving your public key, Dr. Harald Schernthanner (hschernt@uni-potsdam.de) creates an account for you, adds your public key to the system, and contacts you when your account is ready. You will receive your username and the IP address or hostname of your assigned PC Pool machine.

7. When your account is ready, log in from PowerShell or Windows Terminal:

```
ssh -i $env:USERPROFILE\.ssh\pcpool_ed25519 <username>@<assigned_pc_or_ip>
```

Replace `<username>` with your assigned username and `<assigned_pc_or_ip>` with the IP address or hostname of your assigned machine.

8. On your first login, SSH will ask whether you trust the remote host. Check that the hostname or IP address matches the information you received, then type:

```
yes
```

You are now logged in to the remote PC Pool machine.

3.1.1.2 Alternative: Using PuTTY on Windows

Users who prefer a graphical SSH client may use PuTTY. PuTTY includes **PuTTYgen**, a tool for generating SSH key pairs, and **PuTTY**, the SSH client used to connect to remote machines.

1. Install PuTTY on your computer.
2. Open **PuTTYgen**:

   Start menu → search for "PuTTYgen" → open the program
3. In PuTTYgen, generate a new SSH key pair. We recommend using an **Ed25519** key. If Ed25519 is not available, generate an **RSA** key with at least **4096 bits**.
4. Save both the private and public key in a secure location on your personal computer.
5. Copy the contents of the public key and send it by email to the system administrator, Dr. Harald Schernthanner (hschernt@uni-potsdam.de).

   Important: Protect your private key with a strong passphrase, store it securely, and never share it with anyone. Only the public key should be sent to the administrator.
6. After receiving your public key, Dr. Harald Schernthanner (hschernt@uni-potsdam.de) creates an account for you, installs your public key on the PC Pool system, and contacts you when your account is ready. You will receive your username and the IP address or hostname of your assigned PC Pool machine.
7. Open **PuTTY**:

   - Enter the IP address or hostname of your assigned PC Pool machine in the **Host Name** field.
   - Ensure that the connection type is set to **SSH**.
   - (Optional) Save the session by entering a name under **Saved Sessions** and clicking **Save**.
8. Configure your private key:

   - Navigate to **Connection → SSH → Auth → Credentials**.
   - Under **Private key file for authentication**, click **Browse...**

- Select the private key (`.ppk`) you created with PuTTYgen.
- Return to the **Session** category and click **Open**.

9. On the first connection, PuTTY displays the server's host key fingerprint.

   Verify that the hostname or IP address matches the information you received from the administrator and click **Accept** (or **Yes**, depending on your PuTTY version) to continue.

10. Log in using the username provided by the administrator.

    Once authenticated, you are connected to your assigned PC Pool machine and can start working from the Linux command line.

3.1.2 `ssh` Key Generation and Login: Linux / Mac

On Linux or Mac, you should be able to use `ssh` right away from the terminal without installing any additional software.

1. Open a terminal and execute:

```
ssh-keygen
```

2. You will be asked to "Enter file in which to save the key (/home/ben/.ssh/id_rsa):". Here, define a folder and filename (e.g., `/home/yourusername/yourkeyname`). When you are asked to "Enter a passphrase:" leave it blank by pressing enter (we don't necessarily need a passphrase for the public key).
3. This generates two files: First the private key named `yourkeyname` (**keep it in a safe place and do not hand it out**) and second the public key named `yourkeyname.pub`. Send the public key to the administrator Harald Schernthanner (hschernt@uni-potsdam.de).
4. Harald Schernthanner (hschernt@uni-potsdam.de) generates an account for you after receiving your public key via email, puts your key on the system, contacts you when your account is ready, and tells you your username and the IP of your assigned machine.
5. Login with `ssh` from your terminal. You need to enter the *IP address of your assigned machine*. For username `ben` logging onto the assigned computer with IP address `141.89.113.170` (After the `-i` flag you have to define **the path to your private key**):

```
ssh -i /home/ben/yourkeyname ben@141.89.113.170
```

3.2 Data Storage on PC Pool Computers

Do not store your data in your home directory (`/home/student`). Instead use `/DATA` for fast, local storage. You should create your own sub-directory. For the user `ben`, use:

```
mkdir /DATA/ben
```

Note: The `/home` and `/DATA` directories are periodically wiped (not during courses), so they should not be used for long-term storage on the PC Pool.

3.3 File Transfer Between Local and Remote Computers

At times you will need to move files between your local computer and the remote machines in the PC Pool. For instance, after you finish processing a large dataset on the remote computer you may want to transfer the output to your local system to look at the results and create figures. Or, for an `.ipynb` Jupyter notebook that you ran on the server, you may want to transfer it to your local computer after running. This is a common task and there are many options for file transfer between local and remote systems via SSH File Transfer Protocol (SFTP).

Keep in mind: Likely, your download speed will be very fast (i.e., it will be fast to pull files from the PC Pool), but your upload speed will be slow (i.e., you have to be patient if you put data on the PC Pool).

3.3.1 GUI: FileZilla - *Your most likely choice*

You can install FileZilla Client on any OS. This is a straight forward way to copy files back and forth (you see progress and expected up/download times). If you plan to move files back-and-forth continuously, you may want to look into `rsync` or a similar command-line option.

When you open FileZilla for the first time you will need to click the top left icon ("Open the Site Manager" with the little server icons underneath "File"). In the Site Manager window create a new site, call it "pc-pool", and fill out the "General" tab exactly as below (changing the fields to your host IP, username, and SSH private key file location):

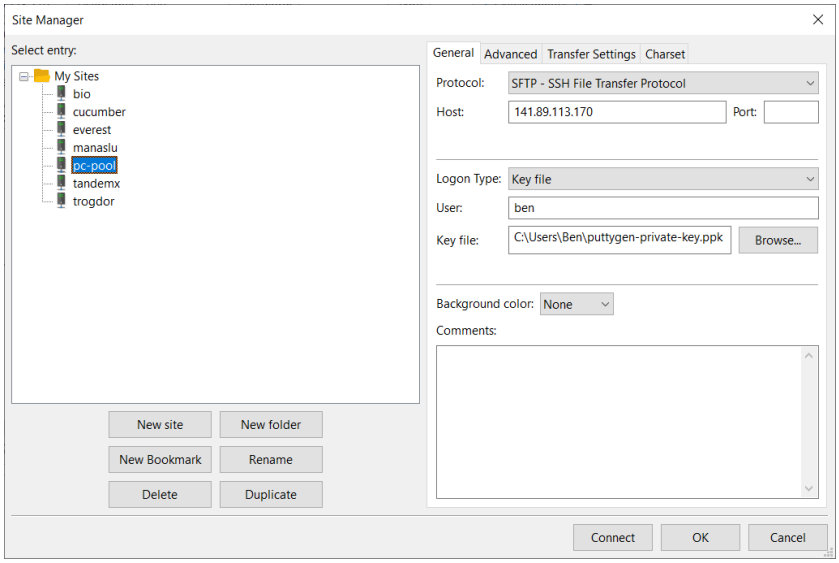

Figure 1: Open a FileZilla session and create a new site to the remote pc-pool host with your username and SSH private key.

After you click "Connect" you can see all of the files on the local and remote systems in two side-by-side panels, navigate around the system folders, and copy data back and forth:

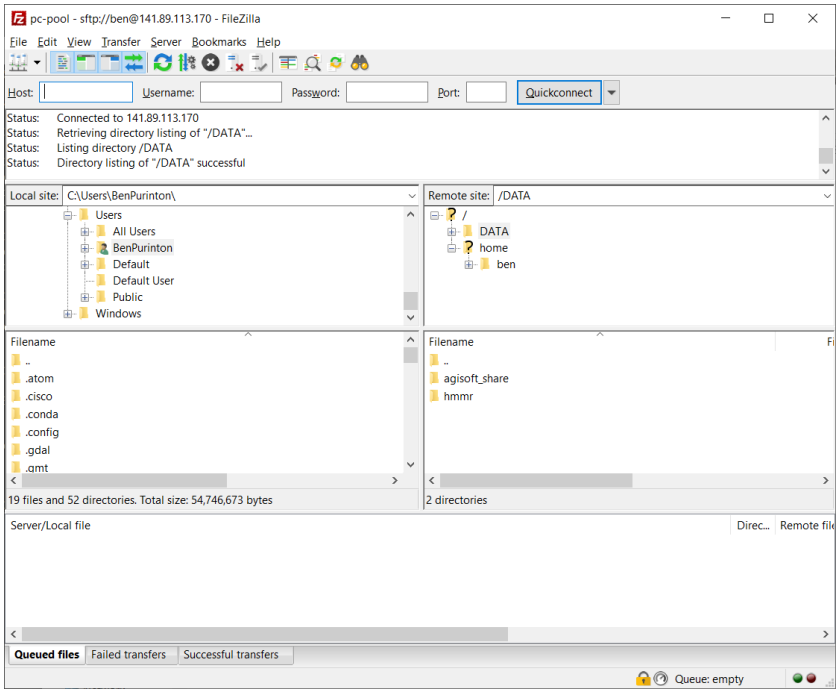

Figure 2: FileZilla connected to remote host showing the file browser for both the local and remote computers.

Note: The new site "pc-pool" will stay in the Site Manager list and can be used to reconnect in the next FileZilla session as long as the SSH private key remains in the same location.

3.3.2 Command Line: `rsync`

If you like to work on the command line (because it is more efficient), rsync is the best option. It should come standard on Linux / Mac. For Windows: it is included in the Cygwin Linux-like terminal; can be added to the Git shell via these instructions; and/or is included in the Windows Subsystem for Linux available in Windows 10.

The basic syntax is:

```
rsync <options> <local filename> <remote filename>
```

For example, to copy the file `my_jupyter_notebook.ipynb` from the local computer to the PC Pool location `/DATA/ben` for user `ben` on computer `141.89.113.160`, use:

```
rsync -avzh --progress my_jupyter_notebook.ipynb ben@141.89.113.160:/DATA/ben/
```

Or to transfer a remote file on the PC Pool to the current directory (**Note:** the period (`.`) below refers to ***current directory***, so wherever you are currently `cd`'d to):

```
rsync -avzh --progress ben@141.89.113.160:/DATA/ben/my_jupyter_notebook.ipynb .
```

The `rsync` options used here include: `-a` (archive), `-v` (verbose), `-z` (compress), `-h` (human readable), and `--progress` (progress bar). You can read about the `rsync` options online via the manual or in cheat sheets.

4 Using JupyterLab Remotely Via SSH

You can run Jupyter on the remote PCs to harness more computing power and to use datasets that you may not want (or be able) to download locally.

4.1 Using `screen` Sessions

When logged in to the remote PC via the ssh command (Mac / Linux) or via PuTTY (Windows), you should *always* start a `screen` session, which keeps your Jupyter notebook or other processes alive even if you lose your local internet connection. Simply enter the command:

```
screen
```

You can familiarize yourself with a few important `screen` commands here. The most important are:

To give the screen session a useful name for finding it later:

```
screen -S screen_name
```

To detach from a screen session without killing the screen (and thus killing the processing), use the keyboard shortcut: `Ctrl-a + d`

And to re-join a detached screen:

```
screen -r screen_name
```

To end a screen, re-join it using the above command and just type `exit`. You can also end a screen without first rejoining it by entering:

```
screen -S screen_name -X quit
```

Note: You should become very familiar with the above screen commands, especially detaching, re-joining, and quitting sessions. These actions should be performed during every SSH session. If new screen sessions are repeatedly started without ending old ones, the remaining processes may eventually need to be terminated by the administrators.

4.2 Opening Remote Notebook Via Port Forwarding: Linux / Mac

On Mac / Linux login to the remote PC using ssh:

```
ssh -i /home/my_username/yourkeyname my_username@my_pc_ip_address
```

Once you are in a **screen session** on the remote PC, JupyterLab can be set up via port forwarding. First do:

```
conda activate <Class Environment Name>
jupyter lab --generate-config
jupyter lab password
```

This will first activate the Python environment for the class (e.g., `DataAnalysis` in place of `<Class Environment Name>`), then generate a configuration file for JupyterLab, and then allow you to create a password to access the JupyterLab session.

Following this, you need to `cd` into the remote directory where your data is stored, e.g., `cd /DATA/my_username/DataAnalysis_Labs/`. Make sure that you have your own directory set up on the `/data` drive! Otherwise you will overwrite each other's work!

Now start the Jupyter Notebook session on the PC Pool:

```
jupyter lab --no-browser --port=XXXX
```

Replace "XXXX" with your assigned port number. _**Note: Use the four-digit port number (e.g., 8888)

Replace "XXXX" with your assigned port number. ***Note: Use the four-digit port number (e.g., 8888) that was assigned to you along with your PC IP address and username. If you accidentally use the same port as someone else on the same PC, you would be trying to edit each other's notebooks!***

Now you are all set on the server side and you can open a new command line / terminal on your local computer.

The next step is to tell your local computer how to access the remotely running JupyterLab session started on the server. This is done via port forwarding. Once again, this is a simple one-line command **run in a new command-line prompt on your local computer**:

```
ssh -N -f -L localhost:YYYY:localhost:XXXX \
    -i /home/my_username/yourkeyname \
    my_username@my_pc_ip_address
```

`-N` tells SSH that no remote commands will be executed, `-f` tells SSH to go to background so the local terminal remains usable, and `-L` lists the port forwarding configuration (remote port XXXX to local port YYYY). Here, "XXXX" is the same port you used in the remote PC command above, and "YYYY" is the port you want your local computer to listen on. You can use the default 8888 in place of "YYYY", or whatever port you prefer. This command does not show any output – it can be checked by opening a web browser and navigating to the URL "http://localhost:YYYY/lab".

You should now be able to access the remote notebooks directly from your browser, while using the computing power of the PC Pool. File transfers between the local and remote machines for datasets, notebooks, and outputs from running scripts can be accomplished using FileZilla or `rsync`, as described above.

4.3 Opening Remote Notebook Via Port Forwarding: Windows

On Windows first open PuTTY and enter the IP address of your assigned machine (**Note:** The IP address can be saved by entering it in the "Saved Sessions" box and then pressing "Save", so it does not need to be entered each time PuTTY is opened).

After this, navigate to Connection > SSH > Tunnels and put in "Source port" the local port that you want to listen on. You can use 8888 here, or any other port greater than 8000 (e.g., 8881). In the "Destination" put "localhost:XXXX" and then press "Add". Replace "XXXX" with your assigned port number. ***Note: Use the four-digit port number (e.g., 8888) that was assigned to you along with your PC IP address and username. If you accidentally use the same port as someone else on the same PC, you would be trying to edit each other's notebooks!***

In this example the user `ben` has been assigned port 8889:

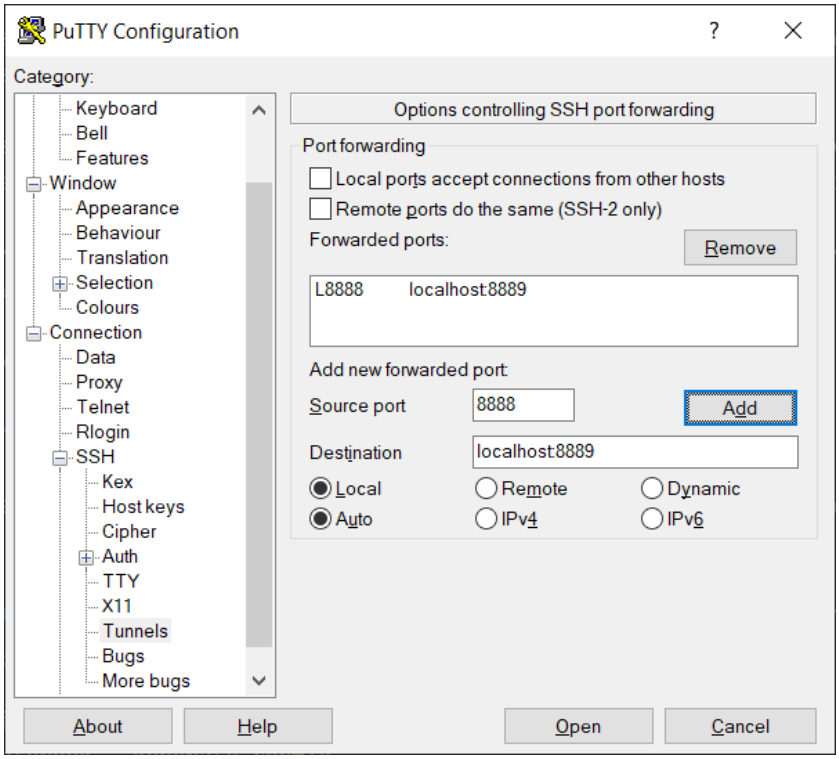

Figure 3: Port forwarding in PuTTY.

Now you can press "Open" to start the remote session with port forwarding setup. Your computer will "listen" on port 8888 and the remote PC will broadcast on port 8889, or whatever port combination you use.

In the remote terminal start a **screen session**. Once the screen session is running on the remote PC, JupyterLab can be set up. First do:

```
conda activate <Class Environment Name>
jupyter lab --generate-config
jupyter lab password
```

This will first activate the Python environment for the class (e.g., `DataAnalysis` in place of `<Class Environment Name>`), then generate a configuration file for JupyterLab, and then allow you to create a password to access the JupyterLab session.

Following this, you need to `cd` into the remote directory where your data is stored, e.g., `cd /DATA/my_username/DataAnalysis_Labs/`. Make sure that you have your own directory set up on the `/data` drive! Otherwise you will overwrite each other's work!

Now start the Jupyter Notebook session on the PC Pool:

```
jupyter lab --no-browser --port=XXXX
```

Replacing "XXXX" with your assigned port number.

Now on your local computer open a browser and navigate to the URL "http://localhost:8888/lab", replacing 8888 with whichever port you put in the "Source port" in PuTTY.

You should now be able to access the remote notebooks directly from your browser, while using the computing power of the PC Pool. File transfers between the local and remote machines for datasets, notebooks, and outputs from running scripts can be accomplished using FileZilla or `rsync`, as described above.